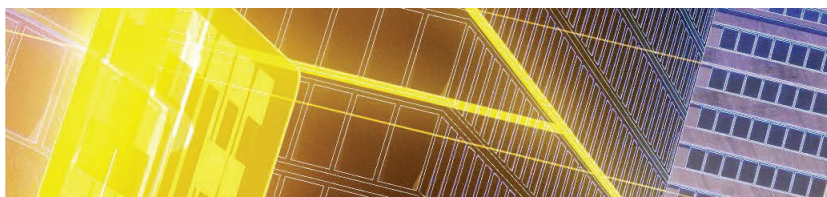


Economic Governance of Software Delivery

Murray Cantor and Walker Royce, IBM

// Agile methods are mainstream, and software enterprises are adopting them in diverse delivery contexts at an enterprise scale. IBM's broad industry experience with agile transformations and deep internal know-how reveals three key principles for delivering sustained improvements in software business outcomes: measure change costs, streamline overhead, and steer with economic governance and Bayesian analytics. //



TODAY'S AGILE PRACTICES are neither novel nor extreme: many practical ideas have been piloted, practiced, and evolved.¹⁻³ What differentiates those enterprises that significantly improve software productivity from those that flounder? One critical factor is an appropriate

governance model that complements the dynamics of agile practices.

Economic governance provides a foundation for quantified planning, decision making, and measuring that resolves uncertainty earlier and unifies constituencies to manage a shared set of expected target

outcomes. Transforming from conventional engineering governance to economic governance changes the principles and perspectives that drive technical and business leadership. An engineering orientation focuses on static targets, planned activities, and deterministic predictions to drive the development process, whereas an economic orientation steers project priorities and results based on the dynamically changing predictions of the probable outcomes of the development process.

Successfully delivering software in a predictable and profitable manner requires an agile leadership style in which leaders should expect an emerging mixture of discovery, production, assessment, and refactoring. Agility is the ability to quickly react or change direction freely and fluidly. Organizations and projects that can exploit agility have an advantage. When changes are less costly to implement, a project is more likely to increase the number of changes, ideally improving quality, efficiency, and timeliness with each iteration. True agility comes when life-cycle changes become more predictable and straightforward over time.

Agility is best measured by quantifying the costs of change over time. When you find teams that have improved their turnaround time with software changes from a few weeks to a few days, you know that they've become more agile. Instead of being mired in late scrap and rework, consumed with playing defense, they can go on the offensive by innovating, adding more features, improving performance, and delivering earlier.

Economic governance is a natural complement to agile development and lean transformation techniques.



Here, we summarize IBM's decade of experience with agile transformations and identify key governance principles for delivering better business outcomes through improvements in software delivery proficiency.

Software Delivery: Economics Trumps Engineering

Successful software outcomes are highly dependent on continuous negotiations, accurate predictions, value judgments, innovations, team collaboration, architects, agility, market conditions, and user demand. Success is much less dependent on contract quality, Gantt charts, critical path schedules, earned value measurement, laws of physics, material properties, mature building codes, and certified engineers. Stated another way, software delivery governance is more a discipline of economics than of engineering. It's a nondeterministic endeavor with much more inherent uncertainty. Entrepreneurial leadership principles and economic governance trump the conventional engineering management doctrine.

Engineering endeavors are generally governed by well-understood laws of physics and properties of materials, as well as centuries of precedent experience. High-uncertainty engineering efforts are still attempted, such as capping the Gulf of Mexico oil leak in 2010, but these are the exceptions. Software delivery, by comparison, is a discipline dominated by human creativity, market forecasting, value judgments, and uncertainty levels similar to economic endeavors such as producing a movie or play,

writing a book, or participating in venture capital management.^{4,5} Consider the following observations:

- Most software activities have no laws of physics or properties of materials to constrain their problems or solutions. They're

This mindset change is well articulated by Eric Ries in *The Lean Startup*.⁵ His approach begins by capturing the business case as a clear hypothesis (a prediction of the value proposition) and then tests that prediction through a sequence of experiments that empirically validate a

Software delivery governance is more a discipline of economics than of engineering.

bound only by human imagination and economic constraints.

- In a software project, you can change almost anything at any time: plans, people, funding, requirements, designs, and tests. Requirements—probably the most misused word in our industry—rarely specify anything that's truly required. Nearly everything is negotiable.
- Quality metrics for software products have few accepted benchmarks. With the possible exception of reliability, most aspects of quality (such as responsiveness, maintainability, and usability) entail vast uncertainty and are still largely measured subjectively, through the eyes of their beholders in the user community.

Most software project managers, movie producers, and entrepreneurs manage teams that create a unique and complex web of intellectual property bounded only by an abstract vision and human creativity.

strategy. When choosing among the many assertions in a business plan, it makes economic sense to test the riskiest assumptions first. This results in the most significant reductions in uncertainty. A minimum viable product constitutes the critical first experiment.

Transforming to Economic Governance

Software enterprises face a significant challenge when they undertake a cultural transformation from the deterministic world of conventional engineering governance to the probabilistic mindset inherent in economic governance. This is a quantum leap for most stakeholders—from static targets to dynamic targets, from managing certainty to managing uncertainty, and from deterministic decisions to probabilistic decisions. This isn't easy for most of us. Table 1 illuminates some of the shifts in governance mindsets. Over the past two decades, we've collectively worked with hundreds of projects mired in engineering governance

TABLE 1

Conventional engineering governance versus modern economic governance.

Engineering governance	Economic governance
Distinct handoff from development team to operations team	Common process, platform, and team for development and operations
Distinct and sequential activities: requirements to design to code to test	Continuous delivery of demonstrable outcomes with ever-increasing value
Role-specific processes and tools	Collaborative platform of integrated, Web-based tools and practices
Early false precision in plans and requirements; focus on the well-understood elements first	Evolving, honest precision as uncertainties are resolved; focus on the big uncertainties first
A static plan that's a sequence of artifacts: specs, models, presentations, test documents, and so on	A dynamic plan that's a sequence of executable, change-controlled releases
Complete unit test coverage prior to integration and system-level testing	Complete integration testing prior to investing in unit test coverage and completeness
Measurement of artifact production and activity completion	Measurement of progress/quality trends in testable releases
Measurement of achieved milestones and variance of actual results to planned targets	Quantified uncertainty, negotiated targets, and steering toward optimized win-win targets
Deterministic engineering discipline: plan and track mindset based on static targets and deterministic predictions	Probabilistic economic discipline: predict, quantify, and steer based on dynamic targets and objective evidence

and experienced perhaps 20 that clearly succeeded with the patterns of economic governance.

Conventional engineering project management techniques assume little uncertainty in their requirements and exploit mature precedents for production and deployment. Software projects managed with such engineering governance models typically uncover stifling, malignant changes late in the life cycle and spend 40 percent or more of their effort consumed in late scrap and rework.⁶ The iron law of traditional software engineering is this: the later you are in the life cycle, the more expensive things are to fix. If you're experiencing this iron law, you're probably anchored in a conventional waterfall-model process. Your process might be mature, or it might more accurately be described as geriatric.

Project managers who are

experienced and trained in traditional project management disciplines such as detailed planning, critical-path analysis, and earned value management have a particularly rough transition. They must move from a world of managing certainty and precision to a world of resolving uncertainty based on imprecise probabilistic judgments.

Although these ideas are far from new, they're also far from being standard practice in most software enterprises. The risk management foundation underlying the spiral model and the basic ideas of software engineering economics were first laid out by Barry Boehm in the 1980s.^{7,8} Applying probability theory to deal with uncertainty is also well established.^{9,10}

A Parable on the Crux of Economic Governance

A short video version of the following parable is available at

https://www.youtube.com/watch?v=ghAM8ifyeVI&feature=youtube_gdata_player.

Suppose you have a software product that needs to be delivered in 12 months to satisfy a critical business need. As the organizational leader, you decide to solicit proposals from two competing project managers. Gerrard Hattrick is a veteran software project manager; Shirley Nimble is a more contemporary software manager. Using empirical estimation models, they both estimate that the project should take 11 months. From here, their approaches diverge.

Gerry, a French-Canadian hockey fan, is certified in traditional project management techniques. His surname compels him to initiate projects with three goals: plan the entire 12 months in detail, specify the requirements precisely, and conduct an early design review to demonstrate

the elements that are easiest to mock up. By demonstrating rapid progress in an early milestone, he can nail down requirements and design decisions and avoid late changes in scope. He's confident that he can deliver a month ahead of schedule.

Shirley sees things differently. She understands that the estimation model's output is simply the mean of a more complex random variable. She asks to see the full distribution of possible outcomes. She wants to go into the project with a 95 percent chance of delivering on time. Her leadership team shows her the baseline estimate at the top of Figure 1.

After assessing the complete distribution, Shirley's team realizes that about half of the outcomes will take longer than 12 months, so they have only a 50-50 chance of delivering on time. There's significant uncertainty in the various input parameters, reflecting the team's lack of certainty about scope, design, plan, and team capability. Consequently, the variance of the distribution of outcomes is rather wide. There are three alternative paths to ensure that 95 percent of the outcomes fall within the target date:

- option 1, ask the sponsor to move out the target delivery date to 15 months;
- option 2, ask the sponsor to rescope the work, eliminating some of the features or backing off on quality so that the median schedule estimate moves up by a month or so; and
- option 3, explicitly reduce the uncertainties in scope, design, plans, team, platform, and process; this narrows the dispersion of the distribution and increases the probability of delivering by the target date.

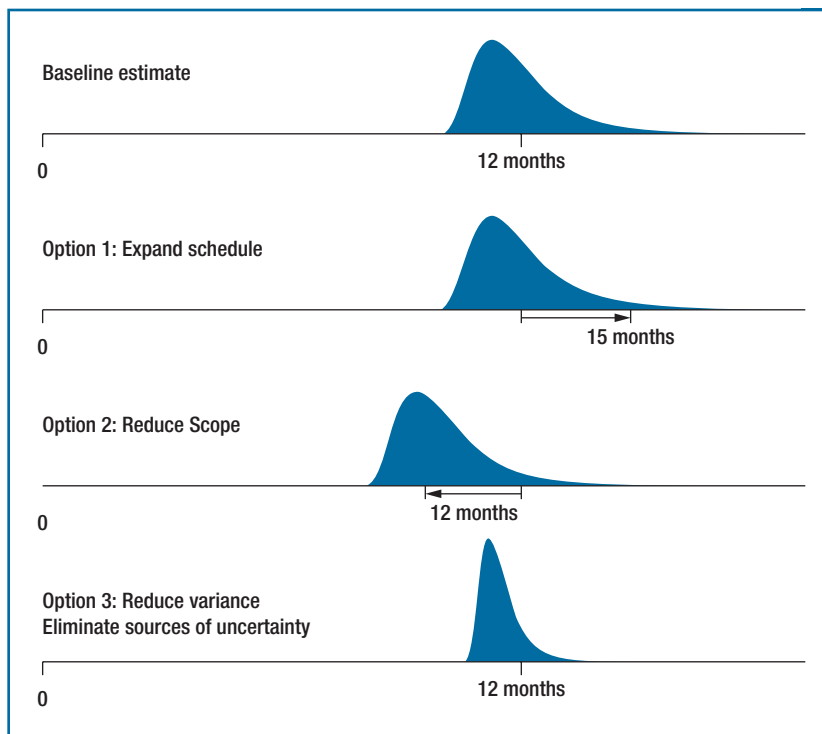


FIGURE 1. A baseline estimate and three options. Option 3 is usually the only acceptable alternative.

The first two options are usually deemed unacceptable by external stakeholders, leaving the third as Shirley's preferred alternative.

Gerry prioritizes his efforts based on what he knows right now. He ignores the big uncertainties, postponing their resolution until after he builds up early project momentum by tackling the straightforward tasks and showing early progress in building the easier, well-understood parts. At first, Gerry's team will appear to be making progress. Late in the project, however, one of the big uncertainties will likely explode into a formidable roadblock, resulting in a late surprise with significant project delay. Gerry Hatrick's methods are geriatric.

Shirley takes the smarter and more courageous approach. She

proceeds through a series of experiments by building early versions to test the design and gain feedback from target users on progress. She demands that integration testing be mostly completed before investing in unit test coverage. These experiments result in validated learning, which is an increased understanding (or decreased uncertainty) in the target system's business case, scope, and design. She resolves the harder challenges first, delivering fewer tangible artifacts early but learning more about what constitutes a successful delivery. She increases the probability of long-term success and decreases the probability of future scrap and rework that add friction and impede a successful delivery. Shirley is more likely to deliver the right software on time.

Gerry's outdated engineering orientation focuses on static targets and deterministic predictions in the face of significant uncertainty. Shirley's economic orientation steers projects with dynamically changing predictions of probable outcomes and a more honest discussion of uncertainties. She's operating like a savvy entrepreneur, striving for and measuring validated learning, as discussed in *The Lean Startup*.⁵ A related discussion about applying options for valuation techniques to decision trees⁹ is an enlightening perspective on quantifying validated learning, as is its earlier foundation work.¹⁰

Bayesian Reasoning

Setting an expected delivery date or planning resources for any project involves a prediction, and the best way to improve predictions is to apply Bayesian reasoning. This type of reasoning is based on centuries-old mathematics and was historically a controversial way of thinking about the nature of probability. It has evolved into an accepted best practice, central to all sorts of predictions. In *The Signal and the Noise*,¹¹ for example, Nate Silver explains how Bayesian reasoning is central to predicting outcomes of elections, economic forecasts, disease propagation, weather,

change of mindset from the way most of us approach project estimation. We must abandon the idea that a project has a fixed duration or a fixed cost. The Bayesian mindset reasons about predictions as a range of possible values called outcomes. Some outcomes are more likely than others, so each is assigned a likelihood value.¹³

The challenge for project leadership is to identify an initial distribution such as time to complete and then continuously update the distribution as the project proceeds. The distribution's changing shape provides insight into whether the probability of meeting the schedule target is improving. The initial distribution is called the *prior* and the updated distribution the *posterior*. In Figure 1, the baseline estimate at the top would be the prior, and the result after pursuing option 3 at the bottom of the figure would be the posterior. The process is rather simple:

- Make an initial, but informed, guess of a prior distribution.
- Gather evidence.
- Use Bayes's theorem to update a posterior distribution.
- Continue updating the posterior distribution as more evidence becomes available.

specification and the perfect plan aren't necessary to improve predictability. It's far better to make a reasonable estimate and then refine that estimate. This is the spirit of iterative and agile development that differentiates it from conventional waterfall-model thinking.

More Honest Predictions

The most likely outcome might be the truth (usually something close to the mean value), but a more honest representation, or the whole truth, would be the full distribution of possible outcomes. For example, the most likely outcome of 11 months is an accurate portrayal of the expected target date depicted for the upper distribution in Figure 1. However, by expressing how sure we are of that guess—by exposing the variance in the distribution—we're much more honest and transparent in communicating that information to others.

The (mathematical) variance of the distribution of outcomes for the estimate to completion can be measured periodically to quantify the trends in uncertainty reduction. Each development phase should reduce uncertainty in the evolving plans, specifications, and demonstrable releases. At any point in the life cycle, the precision of the subordinate artifacts, especially the plans and variance in estimates to completion, should be in balance with the evolving precision in understanding.

Stephen Covey coined the phrase "the speed of trust" to highlight trust as the necessary element in reducing overhead (that is, in becoming leaner) and improving efficiency.¹⁴ The speed of trust can also be appreciated through its logical opposite: the slowness of distrust. Overhead activities in most enterprises are directly proportional to the amount of

The perfect specification and plan aren't necessary to improve predictability.

poker, and other societal interests. Bayesian methods have also been applied to managing project durations in other engineering domains.¹²

Bayesian reasoning requires a

One of the important features of this approach is that it converges to the same posterior no matter what prior was used to seed the process. This is important: the perfect

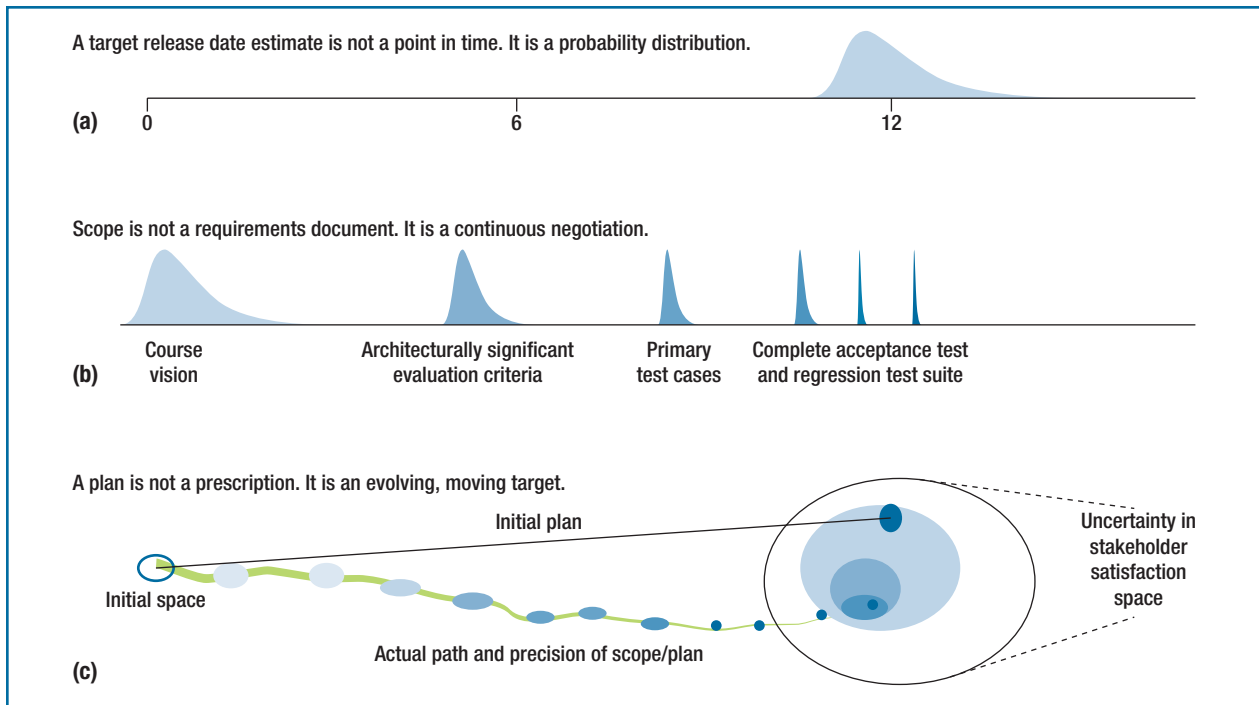


FIGURE 2. Important steering perspectives to manage uncertainty. Precision in specifications, plans, and estimates evolve in balance with one another.

distrust in the organization. Because humans make errors, some distrust is healthy and necessary. However, when overhead activities are overly burdensome, the system becomes inefficient. More honest communications improve trust among stakeholders, which enables greater efficiency. Deterministic planning kills trust because it doesn't match the reality and uncertainties found in the software development world. Probabilistically communicating plans and targets and explicitly quantifying any uncertainty builds trust because it accurately portrays an understanding of how to reason about the future.

All stakeholders must collaborate to converge on moving targets and manage uncertainties, as Figure 2 shows. Software schedules, scope, designs, and plans must be treated as predictions with inherent

uncertainty, not as contracts with implied certainties and requirements. A target date is a prediction—typically, the mean of a probability distribution of the release dates. Requirement specifications and models evolve from a coarse vision with a wide variance to a precise, testable specification with relative certainty. Operational characteristics remain negotiable as trade-offs evolve from speculative debates to factual decisions. A project plan can be captured as the predicted set of state changes en route to the desired end state. If uncertainties have been resolved, life-cycle artifacts should evolve with increased precision.

Iterative, lean, and agile development methods have emerged organically from diverse software development communities to improve navigation through uncertainty.

This steering requires measured improvement with dynamic controls, instrumentation, and intermediate checkpoints that permit stakeholders to assess what they've achieved so far (the "as-is" situation), what adjustments they should make to the target objectives (the "predicted-to-be" situation), and how to refactor what they've achieved to adjust those targets in the most economic way (the road map forward). The key results will be reduced overhead and a significant reduction (perhaps as high as 50 percent) in scrap and rework.^{6,15}

To transform successfully from conventional engineering governance to more agile economic governance requires a significant cultural transformation, best achieved by demanding that integration testing precede unit testing.¹⁵ Integrating first will

expose architecturally significant challenges and help resolve big uncertainties earlier. It's human nature to address the easy things first to show early progress, but economic leverage isn't gained by doing so.

Reasoning and Improved Predictability

Let's return to our parable. Shirley plans to use an agile process, decomposing the features into work items and assigning those work items to a sequence of builds. Before she commits to the project, she develops an initial distribution of the time to complete it. This distribution provides her with a view of the probability of achieving that goal.

Her team estimates the level of effort with simple triangular distributions. For each feature's level of effort, the leadership team agrees on three values:

- The low value (best case) assumes that all the stars align and the feature comes together easily to meet requirements.

more than the worst case, so the distributions are set to zero below the low value and above the high value.

In Bayesian reasoning, this technique provides subject matter experts with a means of arriving at an honest prior based on current information and informed belief. If the difference between the low and high for a feature is large, then the team is expressing high uncertainty in the effort required to deliver that feature. This gives Shirley's team greater opportunity to resolve higher priority uncertainties early on.

With this prior estimate, Shirley has an idea for how likely it is to meet the commitment, and she negotiates the content. Through some analysis, she finds that one of the relatively uncertain features is more of a nice-to-have than a must-have and would add considerably more risk than value. So she negotiates that feature out of scope for a firmer commitment to a delivery in 11 months.

As Shirley's team completes the work items and plan items, they update the time to complete random

whether the odds of meeting the commitment are improving (as they should) or not. With a quantified and objective view of the project trends, all stakeholders can have a more honest and trustworthy discussion about how best to proceed. Discussing the mean estimate is the truth, and discussing the variance in the distribution is the whole truth.

IBM's broad industry experience with thousands of projects over a decade of internal agile transformation experience points to three priorities for delivering sustained improvements in software business outcomes with higher confidence:

- measure change costs,
- steer using economic governance, and
- streamline overhead.

Continuous measurement in executable software baselines is a cornerstone of agility. Measurements must illuminate the progress and quality indicators needed to steer projects to more successful outcomes. The agile principles of continuous integration and test-driven development lay a strong foundation for measured improvement. The core metrics for steering can be extracted from the release baselines undergoing continuous integration testing. These metrics quantify the validated learning for more objective reduction of uncertainty and trustworthy assessment of progress and quality.

Steering projects with economic governance demands different leadership priorities:

- Manage target costs and schedules as narrowing distributions of outcomes.

It's human nature to address the easy things first, but economic leverage isn't gained by doing so.

- The nominal value (most likely) assumes the level of effort has the expected mix of good fortune and bad luck.
- The high value (worst case) assumes that Murphy's law dominates, presenting unexpected challenges and obstacles.

There's no chance that the level of effort will be less than the best case or

variables based on their increased understanding of progress and quality. They use completed work and actual team performance (facts), combined with updated plans to complete (estimates), to update the time to complete distribution (honest predictions) with sound mathematics.

With these periodic predictions, Shirley can discuss with her internal team and external stakeholders

- Predict outcomes using Bayesian reasoning and ever-improving information.
- Optimize quality as a narrowing distribution of defect classes and density.
- Communicate progress and quality trends transparently and honestly.

Software estimates, proposals, and plans are essential predictions that represent the primary information exchanges among governance stakeholders. Trust is earned when integrity and performance are combined. Integrity is improved with more honest predictions that quantify uncertainties; performance is improved by measurably reducing uncertainty early and continuously negotiating and steering.

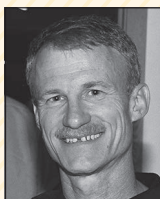
Most of today's software engineering cultures are anchored in deterministic planning and governance methods that compete with practitioner freedom. Management must give practitioners more freedom to innovate through automation of measurement, traceability, progress reports, documentation, and change propagation. And practitioners must provide management with improved steering mechanisms such as progress and control measures and real-time development analytics.

The platform of process know-how and automation must deliver this critical quid pro quo: practitioners must demand that overhead tasks be minimized, streamlined, and automated, and management stakeholders must demand more insightful measures and a feedback control loop to steer progress and quality with better predictability of business outcomes. Trust is the key to unlocking this freedom-for-measurement exchange.

ABOUT THE AUTHORS



MURRAY CANTOR is an IBM Distinguished Engineer and the lead architect for analytics and optimization products at IBM Rational. He was a founding member of the OpenGL architecture review board and is the author of two books: *Object-Oriented Project Management with UML* (John Wiley, 1998) and *Software Leadership* (Addison-Wesley, 2001). Cantor received a PhD in mathematics from the University of California, Berkeley. Contact him at mcantor@us.ibm.com.



WALKER ROYCE is the Chief Software Economist in IBM's Software Group and the author of three books: *Eureka! Discover and Enjoy the Hidden Power of the English Language* (Morgan James, 2011), *The Economics of Software Development* (Addison-Wesley, 2009), and *Software Project Management, A Unified Framework* (Addison-Wesley, 1998). Royce received a BA in Physics from the University of California, Berkeley and an MS in computer engineering from the University of Michigan. Contact him at weroyce@us.ibm.com.

There are probably more books on agile methods than successful projects with well-documented agile results. Writing a book on agility or project management is easy—but managing a real project where you must steer through a minefield of uncertainties is hard. We should all place increased emphasis on publishing measured improvement case studies. They're critical to accelerating the innovation delivered in software. 📖

References

1. J. Highsmith, *Adaptive Software Development*, Dorset House, 1999.
2. M. Poppendieck and T. Poppendieck, *Lean Software Development*, Addison-Wesley, 2003.
3. B. Boehm and Richard Turner, *Balancing Agility and Discipline*, Addison-Wesley/Pearson Education, 2003.
4. R. Austin and L. Devin, *Artful Making*, Prentice Hall, 2003.
5. E. Ries, *The Lean Startup*, Crown Business, 2011.
6. W. Royce, *Software Project Management*, Addison-Wesley, 1998.
7. B. Boehm, "A Spiral Model of Software Development and Enhancement," *Computer*, vol. 21, no. 5, 1988, pp. 61–72.
8. B. Boehm, *Software Engineering Economics*, Prentice Hall, 1981.
9. S. Biffl et al., *Value-Based Software Engineering*, Springer, 2006.
10. H. Erdogmus, "Valuation of Learning Options in Software Development under Private and Market Risk," *Engineering Economist*, vol. 47, no. 3, 2002, pp. 308–353.
11. N. Silver, *The Signal and the Noise: Why Most Predictions Fail and Some Don't*, Penguin Press, 2012.
12. B. Kim and K. Reinschmidt, "Probabilistic Forecasting of Project Duration Using Bayesian Inference and the Beta Distribution," *J. Construction Eng. Management*, vol. 135, no. 3, 2009, pp. 178–186.
13. M. Cantor, "Filling in the Blanks, The Math behind Nate Silver's *The Signal and the Noise*," IBM Developer Works, May 2013; www.ibm.com/developerworks/rational/library/filling-in-the-blanks-1.
14. S. Covey, *The Speed of Trust: The One Thing That Changes Everything*, Free Press, 2006.
15. W. Royce, "Measuring Agility and Architectural Integrity," *Int'l J. Software Informatica*, vol. 5, no. 3, 2011; www.ijsi.org/ch/reader/view_abstract.aspx?file_no=i92.



See www.computer.org/software-multimedia for multimedia content related to this article.